azure-agentic-ai-masterclass

Course Hub - 12-week Master Class

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'azure-agentic-ai-masterclass' - Course Hub - 12-week Master Class. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Design, secure, deploy, observe & operate multi-agent systems on Azure
- One runnable repo per week + framework comparison projects
- Everything runs locally in mock mode; Azure is optional

Notes

[Learning goal] Walk through these talking points:
- Design, secure, deploy, observe & operate multi-agent systems on Azure
- One runnable repo per week + framework comparison projects
- Everything runs locally in mock mode; Azure is optional
Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Audience: AI engineers, solution architects, backend/platform & data teams
- Stack: Python - FastAPI - Microsoft Foundry - Agent Framework - MCP - A2A
- Outcome: the competencies of an Azure AI Architect

Notes

[Enterprise use case] Walk through these talking points:
- Audience: AI engineers, solution architects, backend/platform & data teams
- Stack: Python - FastAPI - Microsoft Foundry - Agent Framework - MCP - A2A
- Outcome: the competencies of an Azure AI Architect
Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Clients -> API Management -> FastAPI orchestration layer
- Agents: Foundry Agent Service - Agent Framework - MCP tools
- Grounding & state: AI Search - Cosmos DB - Redis - Blob
- Cross-cutting: Identity - Observability - Governance
- Deploy: Container Apps / AKS / Functions

Notes

[Architecture / flow] Walk through these talking points:
- Clients -> API Management -> FastAPI orchestration layer
- Agents: Foundry Agent Service - Agent Framework - MCP tools
- Grounding & state: AI Search - Cosmos DB - Redis - Blob
- Cross-cutting: Identity - Observability - Governance
- Deploy: Container Apps / AKS / Functions

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- 12 weekly labs, each a standalone FastAPI service
- Mock backend (offline, tested) + optional real Azure backend
- Plus LangGraph & Semantic Kernel comparison projects
- See COMPARISON.md for framework trade-offs

Notes

[Key concepts] Walk through these talking points:
- 12 weekly labs, each a standalone FastAPI service
- Mock backend (offline, tested) + optional real Azure backend
- Plus LangGraph & Semantic Kernel comparison projects
- See COMPARISON.md for framework trade-offs
Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- Clone any lab repo and open in VS Code
- pip install -r requirements.txt
- uvicorn app.main:app --reload -> /docs
- No Azure needed to learn & demo

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:
- Clone any lab repo and open in VS Code
- pip install -r requirements.txt
- uvicorn app.main:app --reload -> /docs
- No Azure needed to learn & demo
Pause for questions before moving on.

Slide 7 - Architect's takeaways

- Agent vs. workflow vs. single LLM call - choose deliberately
- Mock-first design keeps labs runnable & testable
- Same patterns scale from laptop to production on Azure

Notes

[Architect's takeaways] Walk through these talking points:
- Agent vs. workflow vs. single LLM call - choose deliberately
- Mock-first design keeps labs runnable & testable
- Same patterns scale from laptop to production on Azure
Discussion: when would you NOT choose this approach?